

Shor, we'll do it!

Amey Bhatuse, David González Lociga, José Ernesto Ramos Sánchez and Sergio Castañeiras Morales
(Dated: October 10, 2025)

The advent of classical computation propelled by the birth of information theory has reduced barriers to communication and enhanced access to information. Consequently, protecting public interests has been ensured by using cryptography, a computational domain that relies on the difficulty of classically solving hard problems to secure information. However, the rise of quantum information theory has paved the way for traversing regions of the algorithmic landscape that are unreachable by classical models of computation. Peter Shor at MIT, in his seminal 1994 paper, introduced the first quantum algorithm to factorize large numbers into their respective primes, the basis on which most modern cryptosystems work. Through this report, we summarize the core intuition of Shor's idea and support it with numerical simulation results derived using state-of-the-art quantum computing simulators. First, the mathematical techniques supporting the analytical framework behind the algorithm are outlined followed by the description of a demonstration of the algorithm. Additionally, we provide a brief study of the scaling of the phase estimation precision with respect to the estimation qubit count.

INTRODUCTION DRAFT

DAVID: Quantum Computing is an emerging field based on the theory that illuminates the inherent randomness of quantum physical systems. Under certain considerations, quantum computers can provide exponential complexity separation over some classical computational models [?]. This is often referred to as the quantum advantage, a phenomenon in which quantum computing a task is either impossible for a classical computer or can be done with more precision and less resources using a quantum model of computation. Although the term itself has gained popularity only recently, Richard Feynman first suggested in 1986 that exploiting quantum mechanics could enable simulating quantum systems previously thought of being out of reach for binary bits-based computers [?]. In the early 1990s, several scientists, including Deutsch, Jozsa, Berthiaume, and Brassard, explicitly addressed this idea, showing that while certain tasks could be solved exactly using quantum mechanics, their classical analogues required randomization and could only be solved with some probabilistic guarantee [? ?]. However, these works did not include a problem outside the class of those already known to be solvable in polynomial time on a classical computer with the aid of randomization and small error [1]. In 1994, inspired by Bernstein and Vazirani's work [?], Daniel Simon made an influential contribution by identifying an oracle problem solvable in polynomial time on a quantum computer, but requiring exponential time classically [?], thereby providing a concrete confirmation of Feynman's intuition.

Given these extraordinary findings, the possibility of quantum computing classically hard problems seems compelling, but it is a looming threat to fields like cryptography, where most practical methods often rely on hard-to-solve computational problems. A particular example is the RSA public-key scheme invented in 1978,

which secures systems admitting the difficulty of finding prime number factors of a large integer. The fastest known algorithm for performing this task is the Number Field Sieve with $O(\exp(c(\log n)^{\frac{1}{3}}(\log \log n)^{\frac{2}{3}}))$ complexity [?]. A few years later, Shor [?] introduced an algorithm employing quantum mechanical principles to do the same task in $O((\log n)^2(\log \log n)(\log \log \log n))$ time. We aim to detail this pioneering work in the following sections, starting with a brief outline of the mathematical backbone, followed by a practical implementation and culminating in a short study of the dependence of qubit count on solution precision.

MATHEMATICAL BACKGROUND OF THE ALGORITHM DRAFT

In this section, we address how to reduce the factorization problem to a modular exponentiation problem. **SERGIO:** It is not exactly that... we present the factorization procedure and we see that the bottleneck is finding the order of an element in a group, which can be solved using quantum computing with parallel computation. It is important to note that the problem is not modular exponentiation, since this can be efficiently done classically (in fact we use a function in our Python script which does the modular exponentiation). It is more precise to say that the problem is order finding: Classically we would have to do many modular exponentiations, but this is where the quantum advantage comes and we make use of the parallel computation.

SERGIO:

The goal of the algorithm is the following: given a positive integer n , find $m, t \in \mathbb{N}$, where $m, t > 1$, such that $n = mt$, that is, to decompose n as a product of two non-trivial positive integers. It has been later showed that n can be assumed to be neither odd or a prime power, as for the first case 2 is a trivial factor and an efficient classical algorithm can be found for the latter[? ?]. **SERGIO:** I

would remove the read part, it is unnecessary.

Let $\tilde{m}$ (I rather use another variable, same as Wikipedia *a* or same as article *x*) be a candidate divisor of n , chosen such that $1 < \tilde{m} < n$. Using the Euclidean algorithm, one can compute $\gcd(\tilde{m}, n)$ in polynomial time [?].

If $\gcd(\tilde{m}, n) \neq 1$, then we have found a non-trivial divisor of n . In this case, set $m = \gcd(\tilde{m}, n)$ and $t = \frac{n}{\gcd(\tilde{m}, n)}$.

Otherwise, if $\gcd(\tilde{m}, n) = 1$, then the equivalence class $[\tilde{m}]$ belongs to the multiplicative group, Maybe congruence class is more accurate

$$\left(\frac{\mathbb{Z}}{n\mathbb{Z}}\right)^* = \{[a] \in \frac{\mathbb{Z}}{n\mathbb{Z}} : \gcd(a, n) = 1\}.$$

In this case, we look for the order of $[\tilde{m}]$, namely the smallest positive integer r such that $\tilde{m}^r \equiv 1 \pmod{n}$.

This order r always exists since $\left(\frac{\mathbb{Z}}{n\mathbb{Z}}\right)^*$ is a finite abelian group. The efficient computation of r is precisely the step where the quantum part of the algorithm becomes essential.

Once r is obtained, two cases arise. If r is odd, the choice of $\tilde{m}$ is not useful, and the procedure is repeated with a new guess $\tilde{m}$. If r is even, then

$$(\tilde{m}^{r/2} - 1)(\tilde{m}^{r/2} + 1) \equiv 0 \pmod{n}.$$

Since r is minimal, we know that $\tilde{m}^{r/2} \not\equiv 1 \pmod{n}$. Thus, it follows that $\tilde{m}^{r/2} \equiv -1 \pmod{n}$ i.e. $\tilde{m}^{r/2} + 1$ is a multiple of n . Therefore, we can compute $\gcd(\tilde{m}^{r/2} + 1, n)$. If $\gcd(\tilde{m}^{r/2} + 1, n) \neq 1$, then $\gcd(\tilde{m}^{r/2} + 1, n)$ is a non-trivial divisor of n , and the factorization goal is achieved. ERNESTO: This is WRONG. $\tilde{m}^{r/2} \not\equiv 1 \pmod{n}$ doesn't imply that $\tilde{m}^{r/2} \equiv -1 \pmod{n}$. In fact, it is desired that this does not happen, since we would have to try another $\tilde{m}$. If $\tilde{m}^{r/2} \equiv -1 \pmod{n}$ i.e. $\tilde{m}^{r/2} + 1$ is a multiple of n , when we compute $\gcd(\tilde{m}^{r/2} + 1, n)$ we will obtain n which is not a trivial divisor of n , thus having to try another $\tilde{m}$. Sergio: After mull it over my answer is: You are right! I mean I was really convinced that the wikipedia is the ultimate truth, but it is not the case!!!! Concrete counterexample: $n = 15, a = 2$. Then $2^4 \equiv 1 \pmod{15}$ so $r = 4$. But $2^{r/2} = 2^2 = 4 \not\equiv \pm 1 \pmod{15}$, even though $(4-1)(4+1) = 3 \cdot 5 \equiv 0 \pmod{15}$. Here $\gcd(4-1, 15) = 3$ and $\gcd(4+1, 15) = 5$ give the nontrivial factors. No joke, we could think of reporting the mistake DAVID: Sergio, I am afraid the Wikipedia is never wrong... Explicitly "At this point, it may or may not be the case that $N|a^{r/2} - 1$..."

Otherwise, the procedure is restarted with a new choice of $\tilde{m}$.

QUANTUM FOURIER TRANSFORM DRAFT

This section is entirely based on Nielsen & Chuang [?]. Let $\{|k\rangle\}_{k=0}^{N-1}$ be an orthonormal basis. We define

the Quantum Fourier Transform (QFT) as the unitary operation U such that its action on the basis is

$$U_{QFT} |r\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{i \frac{2\pi}{N} kr} |k\rangle.$$

Considering a system with n qubits, $N = 2^n$ and $\{|k\rangle\}_{k=0}^{N-1}$ is simply the computational basis. We will write the elements of the computational basis as binary strings $k = k_1 k_2 \dots k_n$ with $k_i \in \{0, 1\}$, $\forall i \in \{1, \dots, n\}$, i.e., $k = k_1 2^{n-1} + k_2 2^{n-2} + \dots + k_{n-1} 2^1 + k_n 2^0$. We will also adopt the notation

$$0.k_l k_{l+1} \dots k_m := \frac{k_l}{2} + \frac{k_{l+1}}{4} + \dots + \frac{k_m}{2^{m-l+1}}$$

with $l < m$. By doing so, we can express the QFT on a general basis state as

$$U_{QFT} |k_1 k_2 \dots k_n\rangle = \frac{|0\rangle + e^{2\pi i 0.k_n} |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + e^{2\pi i 0.k_{n-1} k_n} |1\rangle}{\sqrt{2}} \\ \otimes \dots \otimes \frac{|0\rangle + e^{2\pi i 0.k_1 k_2 \dots k_n} |1\rangle}{\sqrt{2}}.$$

This expression enables the implementation of the QFT solely through the composition of two types of quantum gates. The first ones are Hadamard's H and act on a single qubit. They are represented as a matrix by

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

The second ones are controlled rotations CR_k and act in two qubits as

$$CR_k = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{\frac{2\pi i}{2^k}} \end{pmatrix}.$$

Then, the quantum circuit that performs the QFT is the one on Fig. 1. Notice how such implementation is exactly the same as the one proposed by Shor [?].

PRIME FACTORIZATION

As discussed above, given x and n , our goal is to find the minimum integer r such that $x^r \equiv 1 \pmod{n}$ Sergio: We can move to other notations but we must be consistent between sections i.e. $x^r \equiv 1 \leftrightarrow \tilde{m}^2 \equiv 1$. The first thing to do is find $q = 2^t$, a power of 2 for certain $t \in \mathbb{N}$, such that $n^2 < q < 2n^2$. Sergio: Here I would add the comment on: we can discard n from being a power of 2 since there are many efficient classical algorithms to factorize n in these cases. Now, starting from the state given by

$$|0\rangle^{\otimes t} \otimes |0\rangle,$$

we apply a Hadamard gate to each of the first t qubits. This yields

$$\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right)^{\otimes t} \otimes |0\rangle = \frac{1}{\sqrt{2^t}} \sum_{a=0}^{2^t-1} |a\rangle \otimes |0\rangle = \frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} |a\rangle |0\rangle,$$

where the tensorial product has expanded into a sum of all possible 2^t binary strings and we have identified each with an integer $a \in \{0, \dots, 2^t-1\}$. Next, we transform $|0\rangle$ to $|x^a \pmod n\rangle$ using *modular exponentiation*. Sergio: Which a are we talking about?? since $a \in \{0, \dots, 2^m-1\}$. Should be something like: For each a one may compute $|a\rangle |0\rangle \rightarrow |a\rangle |x^a \pmod n\rangle$ using modular exponentiation. This algorithm works as follows. We decompose a as a binary string, i.e., $a = \sum_{i=0}^{l-1} a_i 2^i$ with $a_i \in \{0, 1\}$. Thus

$$\begin{aligned} x^a \pmod n &= x^{\sum_{i=0}^{l-1} a_i 2^i} \pmod n = \prod_{i=0}^{l-1} x^{a_i 2^i} \pmod n \\ &= \prod_{i=0}^{l-1} \left[x^{2^i} \pmod n \right]^{a_i}, \end{aligned}$$

and only when $a_i = 1$, the term is taken into account. Our state is now

$$\frac{1}{\sqrt{q}} |a\rangle |x^a \pmod n\rangle$$

Sergio: Maybe I am misunderstanding something but the state is

$$\frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} |a\rangle |x^a \pmod n\rangle,$$

Absolutely! Forgot the sum :(and we can perform a quantum Fourier transform to the first register $|a\rangle$ yielding

$$\frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} \sum_{c=0}^{q-1} e^{2\pi i a \frac{c}{q}} |c\rangle |x^a \pmod n\rangle.$$

Now we proceed with a projective measurement into a general state $|c\rangle |x^k \pmod n\rangle$ and compute the corresponding probability Sergio: Do not repeat the dummy index i.e. $|c\rangle |x^k \pmod n\rangle \rightarrow |\tilde{c}\rangle |x^k \pmod n\rangle$, or any other letter but c . This is

$$\mathbb{P}(c, x^k \pmod n) = \left| \frac{1}{q} \sum_{\substack{a=0 \\ x^a \equiv x^k \pmod n}}^{q-1} e^{2\pi i a \frac{c}{q}} \right|^2.$$

Without losing focus on our goal, we can introduce r into the equation Sergio: I don't like this sentence, keep it clean, maybe "Now we introduce r into the equation as the following"... or even remove the sentence entirely. By definition $x^{r+a} \equiv x^a \pmod n$, so $x^a \pmod n$ is

periodic with period r . This means that the sum can be reinterpreted as a sum over $a \equiv k \pmod r$ Sergio: It is true, but is more natural to say: Notice that some terms in the sum might be repeated since we can find $|x^i \pmod n\rangle = |x^j \pmod n\rangle$ for $i \neq j$. This happens when

$$x^i \equiv x^j \pmod n \implies i = br + j$$

where we have supposed $i > j$, $b, r \in \mathbb{Z}$ and r satisfies $x^r \equiv 1 \pmod n$ i.e. r is the order of x in $(\frac{\mathbb{Z}}{n\mathbb{Z}})^*$. Therefore, $a = br + k$ for some $k \in \mathbb{Z}$ and the probability is reexpressed as

$$\mathbb{P}(c, x^k \pmod n) = \left| \frac{1}{q} \sum_{b=0}^{\lfloor \frac{q-1-k}{r} \rfloor} e^{2\pi i \frac{c}{q} (br+k)} \right|^2 = \left| \frac{1}{q} \sum_{b=0}^{\lfloor \frac{q-1-k}{r} \rfloor} e^{2\pi i b \frac{rc}{q}} \right|^2.$$

Getting rid of the global unitary factor $e^{2\pi i \frac{c}{q} k}$ in this step looks more logical, then we'll also get rid of the factor $e^{2\pi i b d}$ afterwards. By Euclidean division, $rc = dq + \{rc\}_q$, where $d \in \mathbb{Z}$ and $\{rc\}_q$ is the residue taken in the interval $-\frac{q}{2} < \{rc\}_q \leq \frac{q}{2}$. Then, leaving out the unitary phases, we are left with

$$\begin{aligned} \mathbb{P}(c, x^k \pmod n) &= \left| \frac{1}{q} \sum_{b=0}^{\lfloor \frac{q-1-k}{r} \rfloor} e^{2\pi i b (\frac{\{rc\}_q}{q} + d)} \right|^2 \\ &= \left| \frac{1}{q} \sum_{b=0}^{\lfloor \frac{q-1-k}{r} \rfloor} e^{2\pi i b \frac{\{rc\}_q}{q}} \right|^2. \end{aligned}$$

We can convert the sum into an integral (I really don't understand this step)[?] (Sergio: Don't worry I don't think Shor understands it either;) as

$$\frac{1}{q} \int_0^{\lfloor \frac{q-1-k}{r} \rfloor} db e^{2\pi i b \frac{\{rc\}_q}{q}} + O\left(\frac{\lfloor (q-1-k)/r \rfloor}{q} \left(e^{2\pi i \frac{\{rc\}_q}{q}} - 1\right)\right),$$

where the error term can be bounded by $O(\frac{1}{q})$ if $|\{rc\}_q| \leq \frac{r}{2}$. Under the same condition, the integral term can be shown to be large.

Sergio: I would skip the integral computation at this point (the exact computation is irrelevant), the key is that under some assumptions, an error of $O(\frac{1}{q})$ and assuming $-\frac{r}{2} \leq \{rc\}_q \leq \frac{r}{2}$ it can be shown that

$$\mathbb{P}(c, x^k \pmod n) \approx \frac{\left| \sin\left(\pi \frac{\{rc\}_q}{r}\right) \right|^2}{\pi^2 |\{rc\}_q|^2}.$$

First, the variable change $u = \frac{rb}{q}$ enables one to express the integral as

$$\frac{1}{r} \int_0^{\frac{r}{q} \lfloor \frac{q-1-k}{r} \rfloor} du e^{2\pi i \frac{\{rc\}_q}{r} u} \approx \frac{1}{r} \int_0^1 du e^{2\pi i \frac{\{rc\}_q}{r} u},$$

where we assume an error $O(\frac{1}{q})$ in approximating $\frac{r}{q} \lfloor \frac{q-1-k}{r} \rfloor \approx 1$, since $k < r$. The remaining integral can be directly computed as

$$\frac{1}{r} \int_0^1 du e^{2\pi i \frac{\{rc\}_q}{r} u} = \frac{e^{2\pi i \frac{\{rc\}_q}{r}} - 1}{2\pi i \{rc\}_q} = e^{\pi i \frac{\{rc\}_q}{r}} \frac{\sin\left(\pi \frac{\{rc\}_q}{r}\right)}{\pi \{rc\}_q}.$$

Thus, the probability is given by

$$\mathbb{P}(c, x^k(\text{mod } n)) \approx \frac{\left| \sin\left(\pi \frac{\{rc\}_q}{r}\right) \right|^2}{\pi^2 |\{rc\}_q|^2}.$$

Under the assumption that $\frac{-r}{2} \leq \{rc\}_q \leq \frac{r}{2}$, it is minimized on the boundary of the interval and one obtains an asymptotic lower bound for the probability, which yields $\frac{4}{\pi^2 r^2} \approx 0.405 r^{-2} > \frac{1}{3} r^{-2}$. Therefore the probability will be, at least, $\frac{1}{3r^2}$ as long as $|\{rc\}_q| \leq \frac{r}{2}$. Rearranging this condition by the definition of $\{rc\}_q$ we obtain

$$\left| \frac{c}{q} - \frac{d}{r} \right| \leq \frac{1}{2q}.$$

In this equation we know both c (we measure it) and q , since its the dimension of our QFT.

TBC

FIGURE

This figure must be polished, but it is essentially Shor's algorithm

CIRCUIT IMPLEMENTATION DRAFT

Sergio: Shit happens. Okay, here's a quick summary of the script workflow.

After computing the inverse Fourier transform (I think we only mention the Fourier transform, but not the inverse, which is actually the one we use), we measure the first register, projecting it into the computational basis.

For example, if we have 2 qubits, we get $|0\rangle_A$ and $|0\rangle_B$ with eigenvalue 0, and $|1\rangle_A$ and $|1\rangle_B$ with eigenvalue 1. So the observable looks like:

$$|1\rangle_A \langle 1|_A + |1\rangle_B \langle 1|_B$$

That said, both measurements can be performed separately and then reordered to form a binary number. SUPER IMPORTANT: the bits are reversed compared to the current order, that is, we go from $d-1$ down to 0 if you prefer.

Now, I'm honestly not sure what the most formal way to describe this measurement is. One could say we're measuring the observable:

$$\sum_{i=0}^{q-1} |i\rangle \langle i|$$

where $|i\rangle$ represents the computational basis states written in binary (notice this observable is the identity of the $\otimes_{i=0}^{q-1} \mathcal{H}_i$ hilbert space). This measurement gives us the output of the quantum computation. From this point onward, everything is classical.

The resulting binary numbers are then converted to decimal, and we divide them by q , i.e.

$$\text{output} \rightarrow^{\text{decimal}} \alpha \rightarrow \alpha/q = \alpha/2^t = \beta$$

Here, β is the result of the computation. We then approximate β to the closest fraction γ/δ , with $\gamma, \delta \in \mathbb{N}$ and $\gamma, \delta < N$. (I know you hate my notation, but I don't care :)) **WTF WHY GAMMA AND DELTA ARE NATURALS U ARE FCKIN CRAZY**

We don't really care about the numerator since it can be anything a priori. The interesting part is the denominator, which gives us our beloved r , the order of x in $(\frac{\mathbb{Z}}{n\mathbb{Z}})^*$.

Still, we have to remember that the fraction can simplify. For example, if $r = 4$ and our guess is correct (i.e. $\delta = 4$), we could also have $\gamma = 2$, giving $1/2$ and thus $r = 2$.

So, it's fair to say that each measurement provides a guess for the order of the element. We can always cross-check whether this r is indeed the order, or, more efficiently, just move on and test $x^{r/2} \pm 1$ as potential factors of n (which is probably the smartest move).

Important points to highlight: The number of qubits in the second register must be larger than $\log_2 n$ (Naturally it needs to encode the N therefore you will need at least $\log_2 n$ quwh bits). The more qubits you use in the first register, the better the results (obvious, but still worth mentioning).

CONCLUSIONS AND FUTURE WORK

INTRODUCTION

Quantum Computing is an emerging field based on a theoretical structure built using the inherent randomness of quantum physical systems. If certain conditions are met, quantum computers give an exponential complexity separation against classical computation [?]. This is often referred to as quantum advantage, that is, when a quantum computer can compute a task that classical computers cannot or can do with less precision or more energy. Although the term itself has gained popularity only recently, Richard Feynman first suggested in 1986 that exploiting quantum mechanics could enable more powerful computation than its classical counterpart [?]. In the early 1990s, several scientists, including Deutsch, Jozsa, Berthiaume, and Brassard, explicitly addressed this idea, showing that while certain tasks could be solved exactly using quantum mechanics, their classical analogues required randomization and could only be solved with high probability [? ? ?]. However, these works did not provide a problem outside the class of those already known to be solvable in polynomial time on a classical computer with the aid of randomization and small error [2]. In 1994, inspired by Bernstein and Vazirani's work [?], Daniel Simon made a groundbreaking contribution: he identified an oracle problem solvable in polynomial time on a quantum computer but requiring exponential time classically [?], thereby providing a concrete confirmation of Feynman's intuition.

The possibility of quantum computing classically hard problems is compelling but it is a looming threat to, among other fields, cryptography, as some of its methods are often relied on hard-to-solve computational problems. This is the case for the widely used RSA public-key scheme developed by Rivest, Shamir and Adleman in 1978 [?]. In this system, the security rests on the difficulty of factorizing an integer into prime numbers. In 1995, the fastest known algorithm for factorizing an integer was Number Field Sieve by Lenstra, Lenstra Jr., Manasse and Pollard [?]. Such method takes an asymptotic running time $O(\exp(c(\log n)^{\frac{1}{3}}(\log \log n)^{\frac{2}{3}}))$ to factor an integer n and for some constant c .

In this article, Shor introduces the first quantum algorithm for integer factorization with an asymptotic running time of $O((\log n)^2(\log \log n)(\log \log \log n))$, demonstrating a clear quantum advantage over Lenstra's classical proposal.

MATHEMATICAL BACKGROUND OF THE ALGORITHM

In this section, we present the factorization procedure and we see that the bottleneck is finding the order of an element in a group, which can be solved using quantum computing with parallel computation.

The goal of the algorithm is the following: given a positive integer n , find $p_1, p_2 \in \mathbb{N}$, where $p_1, p_2 > 1$, such that $n = p_1 p_2$, that is, to decompose n as a product of two non-trivial positive integers.

Let x be a candidate divisor of n , chosen such that $1 < x < n$. Using the Euclidean algorithm, one can compute $\gcd(x, n)$ in polynomial time [?].

If $\gcd(x, n) \neq 1$, then we have found a non-trivial divisor of n . In this case, set $p_1 = \gcd(x, n)$ and $p_2 = \frac{n}{\gcd(x, n)}$.

Otherwise, if $\gcd(x, n) = 1$, then the equivalence class $[x]$ belongs to the multiplicative group,

$$\left(\frac{\mathbb{Z}}{n\mathbb{Z}}\right)^* = \{[b] \in \frac{\mathbb{Z}}{n\mathbb{Z}} : \gcd(b, n) = 1\}.$$

In this case, we look for the *order* of $[x]$, namely the smallest positive integer r such that $x^r \equiv 1 \pmod{n}$.

This order r always exists since $\left(\frac{\mathbb{Z}}{n\mathbb{Z}}\right)^*$ is a finite abelian group. The efficient computation of r is precisely the step where the quantum routine of the algorithm becomes essential [since it is a hard computing problem in classical computation i.e. the bottle neck of the algorithm when \$n\$ is a large number.](#)

Once r is obtained, two cases arise. If r is odd, the choice of x is not useful, and the procedure is repeated with a new guess x . If r is even, then

$$(x^{r/2} - 1)(x^{r/2} + 1) \equiv 0 \pmod{n}.$$

It is clear that $x^{r/2} - 1$ and $x^{r/2} + 1$ are potential divisors of n . Therefore, we can compute $\gcd(x^{r/2} + 1, n)$ and/or $\gcd(x^{r/2} - 1, n)$. If $\gcd(x^{r/2} + 1, n) \neq 1$, then $\gcd(x^{r/2} + 1, n)$ is a non-trivial divisor of n , and the factorization goal is achieved, idem for $\gcd(x^{r/2} - 1, n)$.

Otherwise, the procedure is restarted with a new choice of x or taking x as $x^{r/2} \pm 1$.

QUANTUM FOURIER TRANSFORM

[Sergio: we need to motivate the section; something like. One of the key steps of the quantum procedure of the Shor's algorithm is the Quantum Fourier Transform \(QFT\) \[? \], in this section we briefly introduce this unitary transformation and how it is implemented in a quantum circuit.](#)

Let $\{|k\rangle\}_{k=0}^{N-1}$ be an orthonormal basis. We define the *Quantum Fourier Transform* (QFT) [?] as the unitary operation U such that its action on the basis is

$$U_{QFT}|r\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{i\frac{2\pi}{N}kr} |k\rangle.$$

Considering a system with n qubits, $N = 2^n$ and $\{|k\rangle\}_{k=0}^{N-1}$ is simply the computational basis. We will write the elements of the computational basis as binary strings $k = k_1 k_2 \dots k_n$ with $k_i \in \{0, 1\}$, $\forall i \in \{1, \dots, n\}$,

i.e., $k = k_1 2^{n-1} + k_2 2^{n-2} + \dots + k_{n-1} 2^1 + k_n 2^0$. We will also adopt the notation

$$0.k_l k_{l+1} \dots k_m := \frac{k_l}{2} + \frac{k_{l+1}}{4} + \dots + \frac{k_m}{2^{m-l+1}}$$

with $l < m$. By doing so, we can express the QFT on a general basis state as

$$U_{\text{QFT}} |k_1 k_2 \dots k_n\rangle = \frac{|0\rangle + e^{2\pi i 0.k_n} |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + e^{2\pi i 0.k_{n-1} k_n} |1\rangle}{\sqrt{2}} \\ \otimes \dots \otimes \frac{|0\rangle + e^{2\pi i 0.k_1 k_2 \dots k_n} |1\rangle}{\sqrt{2}}. \quad (1)$$

This expression enables the implementation of the QFT solely through the composition of two types of quantum gates. The first ones are Hadamard's (H) and act on a single qubit. They are represented as a matrix by

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

The second ones are controlled rotations (CR_k) and act in two qubits as

$$CR_k = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{\frac{2\pi i}{2^k}} \end{pmatrix}.$$

Then, the quantum circuit that performs the QFT is the one on Fig. 1. Notice how such implementation is exactly the same as the one proposed by Shor [?].

ORDER FINDING

As discussed above the quantum computations in Shor's algorithm are mainly focused in the order finding, i.e. the minimum integer r such that $x^r \equiv 1 \pmod{n}$. Here we introduce implementation of how the computation is implemented in a quantum circuit and discuss the insights and problems of this algorithm.

As discussed above, given x and n , our goal is to find the minimum integer r such that $x^r \equiv 1 \pmod{n}$. The first thing to do is find $q = 2^t$, a power of 2 for certain $t \in \mathbb{N}$, such that $n^2 < q < 2n^2$. Notice we can discard n from being a prime power since there are many efficient classical algorithms to factorize n in these cases [? ?]. Now, starting from the state given by

$$|0\rangle^{\otimes t} \otimes |0\rangle,$$

we apply a Hadamard gate to each of the first t qubits. This yields

$$\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right)^{\otimes t} \otimes |0\rangle = \frac{1}{\sqrt{2^t}} \sum_{a=0}^{2^t-1} |a\rangle \otimes |0\rangle = \frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} |a\rangle |0\rangle,$$

where the tensorial product has expanded into a sum of all possible 2^t binary strings and we have identified each with an integer $a \in \{0, \dots, 2^t - 1\}$. For each a one may compute $|a\rangle |0\rangle \rightarrow |a\rangle |x^a \pmod{n}\rangle$ using modular exponentiation. In particular, the modular exponentiation part of the algorithm works as follows. We decompose a as a binary string, i.e., $a = \sum_{i=0}^t a_i 2^{t-i}$ with $a_i \in \{0, 1\}$. Thus

$$x^a \pmod{n} = x^{\sum_{i=0}^t a_i 2^{t-i}} \pmod{n} = \prod_{i=0}^t x^{a_i 2^{t-i}} \pmod{n} \\ = \prod_{i=0}^t \left[x^{2^{t-i}} \pmod{n} \right]^{a_i},$$

and only when $a_i = 1$, the term is taken into account. Our state is now

$$\frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} |a\rangle |x^a \pmod{n}\rangle,$$

and we can perform a quantum Fourier transform to the first register $|a\rangle$ yielding

$$\frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} \sum_{c=0}^{q-1} e^{2\pi i a \frac{c}{q}} |c\rangle |x^a \pmod{n}\rangle.$$

Now we proceed with a projective measurement into a general state $|\tilde{c}\rangle |x^k \pmod{n}\rangle$ and compute the corresponding probability. This is

$$\mathbb{P}(\tilde{c}, x^k \pmod{n}) = \left| \frac{1}{q} \sum_{\substack{a=0 \\ x^a \equiv x^k \pmod{n}}}^{q-1} e^{2\pi i a \frac{\tilde{c}}{q}} \right|^2.$$

Notice that some terms in the sum might be repeated since we can find $|x^i \pmod{n}\rangle = |x^j \pmod{n}\rangle$ for $i \neq j$. This happens when

$$x^i \equiv x^j \pmod{n} \implies i = br + j,$$

where we have supposed $i > j$, $b, r \in \mathbb{Z}$ and r satisfies $x^r \equiv 1 \pmod{n}$ i.e. r is the order of x in $(\frac{\mathbb{Z}}{n\mathbb{Z}})^*$. Therefore, $a = br + k$ for some $k \in \mathbb{Z}$ and the probability is reexpressed as

$$\mathbb{P}(\tilde{c}, x^k \pmod{n}) = \left| \frac{1}{q} \sum_{b=0}^{\lfloor \frac{q-1-k}{r} \rfloor} e^{2\pi i b \frac{r\tilde{c}}{q}} \right|^2,$$

where we eliminated the global unitary factor $e^{2\pi i \frac{\tilde{c}}{q} k}$. By Euclidean division, $r\tilde{c} = dq + \{r\tilde{c}\}_q$, where $d \in \mathbb{Z}$ and $\{r\tilde{c}\}_q$ is the residue taken in the interval $-\frac{q}{2} < \{r\tilde{c}\}_q \leq \frac{q}{2}$. Such operation spans a phase that we can omit.

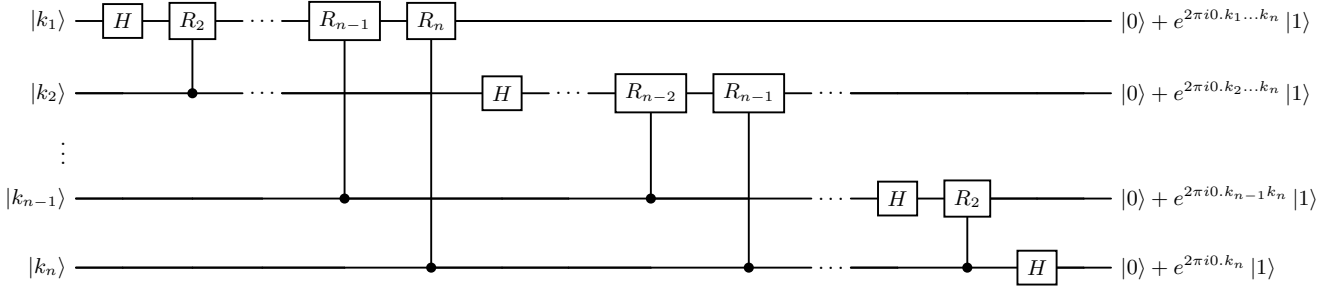


FIG. 1: Quantum circuit implementing the Quantum Fourier Transform (QFT). The output states are shown unnormalized (the $1/\sqrt{2}$ factors are omitted). Controlled rotation gates CR_k are represented by rotation gates R_k acting on the target qubit, with filled dots indicating their respective control qubits. Notice how by reading the output qubits downwards, we obtain the QFT in the reversed order (see Eq. 1), so a final SWAP gate is required in order to obtain the desired result.

With an error of $O(\frac{1}{q})$ and assuming $\frac{-r}{2} \leq \{r\tilde{c}\}_q \leq \frac{r}{2}$ it can be shown that

$$\mathbb{P}(\tilde{c}, x^k(\text{mod } n)) \approx \frac{\left| \sin\left(\pi \frac{\{r\tilde{c}\}_q}{r}\right) \right|^2}{\pi^2 |\{r\tilde{c}\}_q|^2}.$$

Under the assumption that $\frac{-r}{2} \leq \{r\tilde{c}\}_q \leq \frac{r}{2}$, this probability is minimized on the boundary of the interval and one obtains an asymptotic lower bound for the probability, which yields $\frac{4}{\pi^2 r^2} \approx 0.405 r^{-2} > \frac{1}{3} r^{-2}$. Therefore the probability will be, at least, $\frac{1}{3r^2}$ as long as $|\{r\tilde{c}\}_q| \leq \frac{r}{2}$. Rearranging this condition by the definition of $\{r\tilde{c}\}_q$ we obtain

$$\left| \frac{\tilde{c}}{q} - \frac{d}{r} \right| \leq \frac{1}{2q}.$$

In this equation, we know both $\tilde{c}$ (we measure it) and q , since it is the dimension of our QFT. At this point, it can be mathematically proven that, given our restrictions in q , there is at most one solution $\frac{d}{r}$ with $r < n$ to the above inequality. The continued fraction expansion of $\frac{\tilde{c}}{q}$ will find all the best approximations by fractions of this fraction in polynomial time [?], therefore allowing us to find $\frac{d}{r}$ with a certain probability.

Once this fraction is obtained, two possibilities arise. If d and r are relatively prime, i.e. $\gcd(d, r) = 1$, then the denominator r of the reduced fraction directly corresponds to the period we seek. Otherwise, if d and r share a common factor $k > 1$, the continued fraction method yields instead the reduced denominator $r' = r/k$. In this case, we must test integer multiples of r' , such as $2r', 3r'$, and so on, until the true period r is found. This will be seen clearly in our example.

As discussed in Section **Why isn't this working?**, once r is obtained, the quantum routine is finished and a potential divisor can be easily computed classically.

-
- [1] This class is called **BPP**, which stands for bounded-error probabilistic polynomial time.
 - [2] This class is called **BPP**, which stands for bounded-error probabilistic polynomial time.